

Homework 4

Course: RNN and Transformer

Assignment: Multimodal AI – Visual Instruction Tuning (VQA)

Date: Spring 2026

1. Overview

In the final homework before the project, we bridge the gap between Vision and Language. You will work with **Vision-Language Models (VLMs)**, which combine a Vision Encoder (like CLIP or SigLIP) with an LLM (like Vicuna or Llama-3).

Standard VLMs are good at describing general photos (e.g., "a cat on a sofa"), but they often fail at **Domain-Specific VQA** (e.g., reading complex Charts, Dashboard UIs, or Medical X-Rays).

Your task is to take a pre-trained VLM and **Fine-tune** it using **QLoRA (Quantized LoRA)** on a specific dataset to improve its visual understanding capabilities.

- **Core Concepts:** Multimodal Projection, Visual Instruction Tuning, PEFT (LoRA).
 - **Model:** LLaVA-v1.5-7B or Qwen-VL-Chat (Quantized to 4-bit).
-

2. Tasks & Requirements

Part 1: Baseline Inference (20%)

Before training, you must establish a baseline.

1. **Model Setup:** Load a pre-trained VLM (e.g., llava-hf/llava-1.5-7b-hf) using bitsandbytes (4-bit quantization) to fit within the 24GB VRAM.
2. **Zero-Shot Testing:**

- Select 3-5 images from a **specific domain** (e.g., **ChartQA** for charts, or **ROCO** for radiology/medical images).
- Ask the model specific questions (e.g., "What is the value of the blue bar?" or "Is there an abnormality in this X-ray?").
- **Observe:** Does the model hallucinate? Does it give vague descriptions instead of precise answers?
- **Report:** Record these "Before" responses.

Part 2: Visual Instruction Tuning with QLoRA (50%)

Now, you will teach the model to understand this domain.

1. Dataset Preparation:

- Download a small subset (e.g., 500~1000 samples) of a VQA dataset.
- *Recommendation:* [HuggingFaceM4/ChartQA](#) (for chart reading) or a Medical VQA dataset.
- Format the data into the standard VLM conversation format: Image + User Question -> Assistant Answer.

2. Fine-tuning (The Training Loop):

- Use Hugging Face PEFT to apply **LoRA adapters** to the Linear Layers of the LLM (e.g., q_proj, v_proj).
- *Important:* You usually freeze the Vision Encoder (ViT) and only train the adapters on the Language Model.
- Train for 1-3 epochs using the RTX 4090. Monitor the training loss.

3. Technical Requirement: Use SFTTrainer (from trl library) or a custom PyTorch loop that handles the multimodal data collation correctly.

Part 3: Evaluation & Analysis (30%)

Evaluate your fine-tuned adapter.

1. **Inference with Adapter:** Load the base model + your trained LoRA adapter.
 2. **Comparison:** Test on the **same 3-5 images** from Part 1.
 - Did the answer improve?
 - Is the format more consistent?
 3. **Visualization:**
 - In your report, show a side-by-side comparison: **[Image] | [Base Model Answer] | [Fine-tuned Answer]**.
-

3. Submission Guidelines

1. Source Code (GitHub)

- Push your code to GitHub.
- **Repository Structure:**
 - `train.py` / `train.ipynb`: The QLoRA training script.
 - `inference.py`: Script to load the adapter and generate answers.
 - `requirements.txt`: Dependencies (transformers, peft, bitsandbytes, etc.).

2. Report (E3 Platform)

- Submit a PDF named `HW4_StudentID_Name.pdf`.
 - **Content:**
 - GitHub Link.
 - **Loss Curve:** Screenshot of your training loss.
 - **Case Studies:** The side-by-side image/answer comparisons (Part 3).
 - **Discussion:** Why did you choose this specific dataset? What challenges did you face with multimodal data loading?
-

4. Grading Rubric

- **VLM Training Pipeline (40%):** Successfully implemented QLoRA on a VLM.
- **Data Processing (20%):** Correctly formatted image-text pairs for the model.
- **Results & Analysis (30%):** Clear comparison demonstrating model improvement (or analysis of why it failed).
- **Code Quality (10%):** Reproducible and clean code.